

Anforderungsanalyse

Vorlage zum Projekt 1 - Datenanwendung mit Python

Diese Vorlage hilft Ihnen, die wichtigsten Aspekte Ihres Projekts vor Beginn der Programmierung zu klären. Sie ist nicht starr. Wenn Ihnen ein Abschnitt für Ihr Vorhaben unpassend erscheint, ersetzen Sie ihn. Wenn Ihnen etwas fehlt, ergänzen Sie es. Die Anforderungsanalyse ist Ihr Arbeitsdokument - sie wächst und ändert sich mit Ihrem Projekt.

1. Projekt-Steckbrief

Kompakter Überblick. Eine bis zwei Zeilen pro Eintrag genügen.

Projektname	citycheck
Variante	[] A – Strompreis-Assistent [X] B – Reise- und Städte-Dashboard
Autorin / Autor	Kai Elsässer
Erstellt am	02.06.2026
Letzte Änderung	-
Geplanter Tech-Stack	• Backend: ◦ Python v.3.14.5 (latest stable zu Projektbeginn) verwaltet via <i>uv</i> ◦ Datenbank: Sqlite3 (ORM Python-Bibliothek <i>sqlalchemy</i>) ◦ Datenvalidierung und -modellierung via Python-<i>Pydantic</i> ◦ Restful Api via <i>FastApi</i> ◦ Tests via <i>pytest</i> • Frontend: ◦ JavaScript, HTML ◦ stateful Application via <i>HATEOAS</i> (Hypermedia as the Engine of Application State) unter Nutzung von der JS-library <i>htmx</i> • Versionsverwaltung via <i>git</i>

2. Projektziel

Was soll Ihre Anwendung in einem Satz leisten? Zwei bis drei Sätze maximal.

Ein:e Nutzer:in soll in der Lage sein, eine Liste an Lieblingsorten persistent zu speichern zu und editieren.

Für jeden neu angelegten Ort werden die Wetterdaten und wichtige Fakten (Sprache, Währung, etc.) abgerufen und ebenfalls gespeichert.

Beim erneuten Aufruf werden die Daten für alle bereits hinterlegten Orte aktualisiert und zu Verfügung gestellt, auch eine manuelle Aktualisierung kann durch den/die Nutzerin angestoßen werden.

Die Daten werden statistisch aufbereitet visualisiert. (BONUS: Eine Möglichkeit zum Datenexport)

3. Funktionale Anforderungen

Was soll die Anwendung können? Pro Punkt eine Funktion. Stichworte reichen. Versuchen Sie zu unterscheiden zwischen MUSS (Pflicht) und KANN (optional, falls Zeit bleibt).

Muss-Anforderungen

- Nutzerkonto anlegen und verwalten
- Orte suchen und speichern
- Wetter- und weitere Daten für Orte abrufen und speichern
- Daten statistisch aufbereiten und visualisieren

Kann-Anforderungen (Kür)

- Möglichkeit für verschiedene Nutzerkonten per Login
- Datenexport

4. Nicht-funktionale Anforderungen

Eigenschaften der Anwendung jenseits der reinen Funktion: Bedienbarkeit, Antwortzeit, Zuverlässigkeit, Datenschutz, Wartbarkeit. Pro Punkt ein Stichwort plus kurze Erläuterung.

Aspekt	Anforderung in Ihrem Projekt
Bedienbarkeit	
Antwortzeit	
Zuverlässigkeit	
Datenschutz	
Wartbarkeit	

5. User Stories

Beschreiben Sie typische Nutzungssituationen aus Nutzersicht. Format: "Als ... will ich ... damit ich ..." Drei bis fünf User Stories reichen.

Nr.	Als (Rolle)	will ich (Ziel)	damit ich (Nutzen)
-----	-------------	-----------------	--------------------

1			
2			
3			
4			
5			

6. Abgrenzung

Was wird die Anwendung explizit NICHT können? Diese Auflistung schützt Sie vor sich selbst - es ist die Erlaubnis, bestimmte Features bewusst NICHT zu bauen.

- ...
- ...
- ...

7. Datenmodell und Datenquellen

Welche Daten verarbeitet Ihre Anwendung? Woher kommen sie? Wohin gehen sie?

Externe Datenquellen (APIs)

API-Name	URL / Endpunkt	Verwendete Felder
Open-meteo	• https://api.open-meteo.com/v1/forecast •	
	https://archive-api.open-meteo.com/v1/archive	
	https://seasonal-api.open-meteo.com/v1/seasonal	
	https://air-quality-api.open-meteo.com/v1/air-quality	
	https://geocoding-api.open-meteo.com/v1/search	
RestCountries		

Eigene Datenhaltung (Datenbank / Excel)

Geplante Speicherform: Datenbank, sqlite

Geplante Tabellen / Bereiche:

...

8. Datenschutz und Datensicherheit

Welche Daten könnten potenziell schutzwürdig sein? Wie schützen Sie sie? Stichworte zu: Speicherort, API-Schlüssel, Eingabevalidierung, Datensparsamkeit.

Aspekt	Maßnahme in Ihrem Projekt
Welche schutzwürdigen Daten gibt es?	Personenbezogene
Wo werden sie gespeichert?	
Wie sind API-Schlüssel geschützt?	-
Wie werden Eingaben validiert?	
Werden Daten an Dritte übertragen?	

9. Risiken

Was könnte schiefgehen? Technische Risiken, Zeitrissen, fachliche Lücken. Pro Risiko: Was ist das Risiko? Wie wahrscheinlich ist es? Was tun Sie dagegen?

Risiko	Wahrscheinlichkeit	Gegenmaßnahme

--	--	--

10. Architektur-Skizze (in Stichworten)

Welche Module planen Sie? Welche Aufgabe hat jedes Modul? Wie wirken sie zusammen? Eine grobe Liste reicht - eine richtige Skizze als Bild können Sie später anhängen.

Modul	Aufgabe
main.py	Einstiegspunkt der Anwendung

11. Reflexion am Projektende

Diesen Abschnitt füllen Sie am letzten Tag aus - er ist die Grundlage für Ihre Präsentation. Vergleichen Sie ehrlich, was Sie geplant hatten und was tatsächlich entstanden ist.

Was wurde umgesetzt?

Was wurde nicht umgesetzt – und warum?

Wichtigste Designentscheidungen mit Begründung

Stärken des Ergebnisses

Schwächen / Lehren für das nächste Mal

Potenziale – was wäre der nächste sinnvolle Schritt?

Diese Vorlage darf frei angepasst werden. Sie ist ein Hilfsmittel, kein Selbstzweck.